

Database Architecture

Document: docs/architecture/DATABASE_ARCHITECTURE.md

System: HackNest — Student Operating System

Status: Foundational / Authoritative — resolves Critical finding from

ARCHITECTURE_AUDIT_REPORT.md (Section 1, Item 1)

Type: Architecture Specification Only — No SQL, No Migrations, No Implementation

This document follows README.md's index and MASTER_ARCHITECTURE.md's governance exactly. Per MASTER_ARCHITECTURE.md Section 9 (Cross-Document Precedence), this document is the **sole authoritative owner** of: domain data shape, relationships, RLS strategy, storage domain ownership, migrations, and the audit log record specification. Every other document that references any of these must cite this document rather than restate it.

0. Relationship to Governing Documents

- **Vision alignment:** the domain model below exists to serve the vision in MASTER_ARCHITECTURE.md Section 1 — a unified Student Operating System, not a collection of disconnected feature databases.
- **Domain list authority:** MASTER_ARCHITECTURE.md Section 4 deferred the canonical domain enumeration to this document. The list in Section 4 below is now that canonical enumeration. Any subsystem document that previously listed domains (API_ARCHITECTURE.md , SECURITY_ARCHITECTURE.md , TESTING_STRATEGY.md , DEPLOYMENT_ARCHITECTURE.md , IMPLEMENTATION_ROADMAP.md) should be checked against this list on its next revision; discrepancies found during this authoring pass are noted in the Validation Report.
- **Role alignment:** this document uses the canonical role set frozen in MASTER_ARCHITECTURE.md Section 5 (student , moderator , admin , service) wherever role-scoped access is described.

1. Database Design Principles

- **PostgreSQL via Supabase.** The relational model is the primary source of truth; document-shaped or denormalized data is used only where a specific domain's access pattern justifies it, never as a default.
 - **Normalized by default.** Data is normalized to avoid duplication and update anomalies; deliberate denormalization is called out explicitly per-domain where read performance justifies it (see Section 12).
 - **UUID primary keys.** See Section 7.
 - **Foreign key integrity.** Every relationship between domains is enforced by a real foreign key constraint at the database level, not merely assumed at the application layer.
 - **Soft delete by default.** See Section 8.
 - **Every table carries audit fields.** See Section 9.
 - **Every table carries consistent timestamp conventions.** See Section 10.
 - **Performance-first indexing.** Every access pattern a domain is known to need is matched by a deliberate index; indexes are not added reactively after a production slowdown as the default posture.
 - **Secure by default.** Every table containing student or organizational data is protected by Row Level Security from the moment it is created — there is no "add RLS later" step (see Section 19).
-

2. Domain-Driven Data Model

The database is organized around the same domain boundaries defined in `MASTER_ARCHITECTURE.md` Section 4, not around technical convenience groupings. Each domain:

- Owns a bounded set of entities responsible for one coherent concern (e.g., Teams owns team formation and membership; it does not own project data, even though projects belong to teams).
- Exposes its data to other domains only through the relationships explicitly defined in Section 5 — no domain silently reaches into another's entities.
- Is independently reasoned about for lifecycle (Section 14), retention (Section 15), and RLS (Section 19), even though many domains

compose together in practice (e.g., a Hackathon submission touches Hackathons, Teams, and Projects at once).

3. Entity Ownership

Ownership follows two independent dimensions, both of which apply to every entity:

1. **Domain ownership** — which domain's specification governs the entity's shape and lifecycle (see Section 4's domain table).
2. **Identity ownership** — which actor (a specific student, a team, an organization, or the system itself via the `service` role) is the resource owner for RLS and authorization purposes (see Section 19).

These two dimensions are independent: a Project entity is domain-owned by the Projects domain, but identity-owned by whichever Team or individual student created it. Authorization checks use identity ownership; specification and lifecycle questions use domain ownership.

4. Complete Entity List (Domains)

This is the canonical domain enumeration referenced by every other document in `docs/architecture/`.

Authentication

- **Owner:** Authentication domain (backed by Supabase Auth)
- **Purpose:** Owns credential verification, session issuance, and identity establishment. Does not own profile or student-specific data.
- **Relationships:** One-to-one with a Students record (an authenticated identity has at most one Student profile).
- **Lifecycle:** Created at sign-up; updated on credential change; never hard-deleted while any dependent record exists (see Section 8).
- **Dependencies:** None (foundational).

Students

- **Owner:** Students domain
- **Purpose:** Owns core student identity: enrollment status, program, cohort, and university affiliation.
- **Relationships:** One-to-one with Authentication; many-to-one with Universities; one-to-one with Student Profiles.
- **Lifecycle:** Created on first authenticated sign-up completion; soft-deleted on account deletion request.
- **Dependencies:** Authentication, Universities.

Universities

- **Owner:** Universities domain
- **Purpose:** Owns institution records and the association of Students and Organizations to an institution.
- **Relationships:** One-to-many with Students; many-to-many with Organizations (an organization may be affiliated with multiple universities, e.g., a multi-campus club).
- **Lifecycle:** Created and administered centrally (typically by `admin` role); rarely deleted, only deprecated.
- **Dependencies:** None (reference domain).

Student Profiles

- **Owner:** Profiles domain (Student Profile subtype)
- **Purpose:** Owns the public-facing representation of a student: display identity, visibility settings, profile completeness.
- **Relationships:** One-to-one with Students; many-to-many with Skills.
- **Lifecycle:** Created alongside the Students record; updated freely by the owning student; soft-deleted with the Students record.
- **Dependencies:** Students, Skills.

AI Builder Profiles

- **Owner:** Profiles domain (AI Builder Profile subtype)
- **Purpose:** Owns the builder-specific extension of a profile — workspace history summary, generated-artifact showcase linkage — distinct from general Student Profile data.

- **Relationships:** One-to-one with Student Profiles; one-to-many with AI Builder Workspace sessions (owned by the AI Builder Workspace domain at the API layer, referenced here for data linkage only).
- **Lifecycle:** Created on first AI Builder Workspace use; updated as workspace activity accumulates.
- **Dependencies:** Student Profiles.

Skills

- **Owner:** Skills domain
- **Purpose:** Owns the canonical, governed list of skill tags usable across Student Profiles and Team/Project listings.
- **Relationships:** Many-to-many with Student Profiles, Teams (skills sought), and Projects (skills used).
- **Lifecycle:** Centrally administered (governed vocabulary, not free text); rarely deleted, only deprecated to avoid breaking existing associations.
- **Dependencies:** None (reference domain).

Teams

- **Owner:** Teams domain
- **Purpose:** Owns team formation, identity, and team-level visibility/discovery settings.
- **Relationships:** One-to-many with Team Members; one-to-many with Projects; many-to-many with Skills (sought).
- **Lifecycle:** Created by a student (who becomes the initial owner); soft-deleted on dissolution.
- **Dependencies:** Students.

Team Members

- **Owner:** Teams domain
- **Purpose:** Owns the membership relationship and role between a Student and a Team.
- **Relationships:** Many-to-one with Teams; many-to-one with Students.
- **Lifecycle:** Created on invite acceptance or join; hard-removed on leave/removal (membership itself is not soft-deleted — see Section 8

exceptions), while the historical fact of past membership is preserved via Audit Logs, not via the membership record itself.

- **Dependencies:** Teams, Students.

Projects

- **Owner:** Projects domain
- **Purpose:** Owns project records, lifecycle state (draft/active/archived), and showcase presentation.
- **Relationships:** Many-to-one with Teams; many-to-many with Hackathons (a project may be submitted to multiple hackathons over time, though typically one at a time); one-to-many with Project Members (for solo-contributor attribution distinct from full Team membership).
- **Lifecycle:** Created by a Team or individual student; state-transitioned through draft/active/archived; soft-deleted on removal.
- **Dependencies:** Teams, Students.

Project Members

- **Owner:** Projects domain
- **Purpose:** Owns fine-grained contributor attribution on a Project, which may be a subset of or distinct from full Team membership (e.g., a non-team collaborator credited on one project).
- **Relationships:** Many-to-one with Projects; many-to-one with Students.
- **Lifecycle:** Created on project creation or contributor addition; removed on contributor removal.
- **Dependencies:** Projects, Students.

Hackathons

- **Owner:** Hackathons domain
- **Purpose:** Owns hackathon event records, lifecycle (upcoming/registration/submission/judging/completed), and administration.
- **Relationships:** Many-to-one with Organizations (administering sponsor/host); many-to-many with Projects (submissions); many-to-many with Teams (registrations).

- **Lifecycle:** Created by an Organization admin; state-transitioned through its defined lifecycle; archived (not deleted) after completion, for historical/showcase value.
- **Dependencies:** Organizations.

Organizations

- **Owner:** Organizations domain
- **Purpose:** Owns non-university organizational entities (sponsors, clubs, companies) and their administrative membership.
- **Relationships:** Many-to-many with Universities (affiliation); one-to-many with Hackathons (as host/sponsor); one-to-many with Organization Members (administrative role holders).
- **Lifecycle:** Created by an authorized requester, approved per the governance process in `MASTER_ARCHITECTURE.md` Section 11; soft-deleted on deactivation.
- **Dependencies:** None (top-level domain), optionally Universities.

Communities

- **Owner:** Communities domain
- **Purpose:** Owns informal, interest- or affiliation-based groupings of students distinct from formal Teams or Organizations (e.g., a shared-interest group not tied to a single project).
- **Relationships:** Many-to-many with Students (membership); optionally many-to-one with Universities or Organizations (affiliation).
- **Lifecycle:** Created by a student or admin; soft-deleted on dissolution.
- **Dependencies:** Students.

Chat

- **Owner:** Chat domain
- **Purpose:** Owns conversation/channel-level constructs: creation, membership, and participant permissions for a real-time conversation container.
- **Relationships:** One-to-many with Messages; many-to-many with Students (participants); optionally many-to-one with Teams or Projects (scoped conversations).

- **Lifecycle:** Created when a conversation is initiated; soft-deleted on archival; hard-deleted only per the retention policy in Section 15.
- **Dependencies:** Students; optionally Teams, Projects.

Messages

- **Owner:** Messages domain
- **Purpose:** Owns individual message records within a Chat container: content, edit history, deletion state, delivery metadata.
- **Relationships:** Many-to-one with Chat; many-to-one with Students (sender).
- **Lifecycle:** Created on send; soft-deleted on user deletion (content redacted, record retained for moderation/audit purposes per Section 15).
- **Dependencies:** Chat, Students.

Notifications

- **Owner:** Notifications domain
- **Purpose:** Owns notification generation records, delivery preference state, and read/unread status.
- **Relationships:** Many-to-one with Students (recipient); polymorphic reference to the triggering domain entity (e.g., a Team invite, a new Message) — see Section 6 for how this reference is structured without a rigid single foreign key.
- **Lifecycle:** Created on trigger event; marked read/unread by the recipient; hard-deleted per retention policy once sufficiently aged (notifications are lower-sensitivity, higher-volume records; see Section 15).
- **Dependencies:** Students; polymorphically, any domain capable of triggering a notification.

Search

- **Owner:** Search domain (indexing ownership — see Section 21)
- **Purpose:** Owns the query-time composition contract across domains; does not own the underlying source data of any domain it indexes.
- **Relationships:** Read-only, derived relationship to Students, Teams, Projects, Hackathons, Organizations (the searchable domains).

- **Lifecycle:** Index entries are derived and kept in sync with source-domain writes; never a primary source of truth for any field.
- **Dependencies:** Students, Teams, Projects, Hackathons, Organizations.

Activity Feed

- **Owner:** Feed domain
- **Purpose:** Owns feed composition and retrieval — a derived, read-mostly view over events from other domains, not a new source of truth.
- **Relationships:** Read-only, derived relationship to Teams, Projects, Hackathons, and any other event-producing domain.
- **Lifecycle:** Feed entries are generated from source-domain events; aged out per retention policy (Section 15) without affecting the source records they were derived from.
- **Dependencies:** Teams, Projects, Hackathons (as event sources).

Achievements

- **Owner:** Achievements domain
- **Purpose:** Owns badge/milestone records recognizing student accomplishments (e.g., hackathon completion, project milestones).
- **Relationships:** Many-to-one with Students; polymorphic reference to the triggering accomplishment (a Hackathon completion, a Project milestone).
- **Lifecycle:** Created on achievement criteria being met (system-triggered, `service` role); immutable once created (achievements are not editable, only revocable under exceptional, audited circumstances).
- **Dependencies:** Students; polymorphically, Hackathons, Projects.

Bookmarks

- **Owner:** Bookmarks domain
- **Purpose:** Owns a student's saved references to other entities (a saved Project, a saved Hackathon, a saved Team).
- **Relationships:** Many-to-one with Students; polymorphic reference to the bookmarked entity.
- **Lifecycle:** Created on save; hard-deleted on unsave (bookmarks are low-sensitivity, ephemeral-by-nature records, an intentional exception to the

soft-delete default — see Section 8).

- **Dependencies:** Students; polymorphically, any bookmarkable domain.

Files

- **Owner:** Files domain (storage ownership — see Section 20)
- **Purpose:** Owns file metadata records (not the binary content itself, which lives in Object Storage per `DEPLOYMENT_ARCHITECTURE.md`'s Infrastructure section) — filename, type, size, owning entity, upload timestamp.
- **Relationships:** Polymorphic reference to the owning entity (a Student Profile avatar, a Project's media, an Organization's asset, a general Document).
- **Lifecycle:** Created on successful upload (post-validation, per `SECURITY_ARCHITECTURE.md`'s File Upload Security controls); soft-deleted on removal; the underlying object is only purged from storage after the retention grace period in Section 15.
- **Dependencies:** Polymorphically, any domain that accepts uploads (Student Profiles, Projects, Organizations).

Admin

- **Owner:** Admin domain
- **Purpose:** Owns administrative action records and the elevated views/operations available to the `admin` role. Does not duplicate the data of the domains it administers — it operates on them through their own defined structures, using the elevated Service Role access pattern defined in `SECURITY_ARCHITECTURE.md`.
- **Relationships:** No new entities beyond an Admin Action Log, which is a specialization of Audit Logs (Section 9) scoped to admin-tier actions.
- **Lifecycle:** Admin action records are created on every admin operation; never deleted (permanent audit trail).
- **Dependencies:** Every domain (by nature of administrative oversight), Audit Logs.

Analytics

- **Owner:** Analytics domain (data ownership — see Section 22)

- **Purpose:** Owns aggregate, non-individually-identifying metrics derived from other domains' activity.
- **Relationships:** Read-only, derived, and aggregated relationship to all activity-producing domains.
- **Lifecycle:** Aggregate records are computed on a defined cadence; raw underlying events are never duplicated into the Analytics domain's own storage (it computes over source domains' data, consistent with Privacy by Design).
- **Dependencies:** All activity-producing domains, in aggregate only.

Audit Logs

- **Owner:** Audit Logs domain — **the sole authoritative owner of the audit log record specification**, resolving the ownership ambiguity identified in `ARCHITECTURE_AUDIT_REPORT.md` Section 6.
- **Purpose:** Owns the permanent, immutable record of sensitive and state-changing actions across every domain. See Section 9 for the full record specification.
- **Relationships:** Polymorphic reference to the actor (a Student, or `service / admin` identity) and the target entity (any domain's entity).
- **Lifecycle:** Created on every qualifying action; immutable and never deleted, subject only to the retention policy in Section 15 (which, for audit logs, is long-retention by design, not short).
- **Dependencies:** Every domain (as a universal cross-cutting concern), consistent with `SECURITY_ARCHITECTURE.md`'s and `API_ARCHITECTURE.md`'s references to it (both of which describe *behavior around* audit logging but defer the record specification to this document, per `MASTER_ARCHITECTURE.md` Section 9).

Settings

- **Owner:** Settings domain
- **Purpose:** Owns per-student account, privacy, and notification preference state.
- **Relationships:** One-to-one with Students.
- **Lifecycle:** Created alongside the Students record with sensible defaults; updated freely by the owning student; soft-deleted with the Students record.

- **Dependencies:** Students.
-

5. Relationships Between Entities

- **One-to-one** relationships are used only where an entity is a genuine extension of another with no independent lifecycle (e.g., Students ↔ Student Profiles, Students ↔ Settings). A one-to-one relationship's dependent side never outlives its parent.
 - **One-to-many** relationships are the default for ownership hierarchies (e.g., Teams → Team Members, Hackathons → hosted-by Organization). The "many" side always carries the foreign key to the "one" side.
 - **Many-to-many** relationships (e.g., Skills ↔ Student Profiles, Organizations ↔ Universities) are always mediated by an explicit junction entity — never modeled as an array/JSON column masquerading as a relationship, so that referential integrity (Section 6) and indexing (Section 12) apply normally.
 - **Junction table conventions:** a junction entity is named for the two domains it connects (e.g., a Student-to-Skill junction), carries its own audit fields (Section 9) where the relationship itself has meaningful lifecycle (e.g., "when was this skill added"), and carries at minimum the two foreign keys plus any relationship-specific attributes (e.g., a Team Member's role is an attribute of the Team ↔ Student junction, not a separate lookup).
 - **Polymorphic references** (used by Notifications, Bookmarks, Files, Audit Logs, Achievements) are used only where an entity must legitimately refer to "any one of several domains," and are always paired with a discriminator indicating which domain the reference targets — never a bare foreign-key-like column with implicit, undocumented meaning.
-

6. Primary and Foreign Key Strategy

- Every entity has a single-column primary key (see Section 7 for the UUID strategy) — composite primary keys are avoided except for pure junction entities where the combination of both foreign keys is genuinely the natural key.

- Every foreign key enforces referential integrity at the database level; an entity may never reference a nonexistent parent.
 - Foreign keys to a soft-deletable parent (Section 8) do not prevent the parent from being soft-deleted — soft delete is a status change, not a removal, so referential integrity is preserved.
 - Polymorphic references (Section 5) are the one deliberate exception to "always a real foreign key" — since a single column cannot reference multiple tables at the database level, these are validated at the application/API layer (per `API_ARCHITECTURE.md`) in addition to being constrained by the discriminator value.
-

7. UUID Strategy

- Every entity uses a UUID as its primary key, generated at creation time — never a sequential integer, to avoid enumeration and to allow identifiers to be generated client-adjacent (e.g., optimistically) without a round-trip when architecturally useful.
 - UUIDs are treated as opaque identifiers; no domain logic infers meaning from a UUID's value or generation method.
 - Foreign keys always reference the parent's UUID primary key directly, never a secondary "natural" identifier (e.g., a Student's UUID is used in foreign keys, not their email).
-

8. Soft Delete Strategy

- **Default:** every entity is soft-deleted (a status/timestamp marks the record as removed) rather than physically removed, preserving referential integrity and audit history.
- **Explicit exceptions**, each justified by the entity's own low-sensitivity, high-churn nature: **Team Members** (removal is hard-deleted; historical membership is preserved via Audit Logs instead) and **Bookmarks** (hard-deleted on unsave, as an ephemeral, non-sensitive preference record).
- Soft-deleted records are excluded from all default queries and from RLS-permitted access (Section 19) as though they do not exist, except for

Admin-tier and Audit Log access, which may reference them for compliance purposes.

- Soft-deleted records are eligible for eventual hard deletion per the retention policy (Section 15) — soft delete is a grace period, not permanent retention.

9. Audit Fields

Every entity, regardless of domain, carries the following audit fields as a baseline (exact field naming is a `CODING_STANDARDS.md` concern once produced; this section specifies the concepts each entity must carry):

- **Created by** — the identity (Student, or `service`) that created the record.
- **Created at** — see Section 10.
- **Last updated by** — the identity that most recently modified the record.
- **Last updated at** — see Section 10.
- **Deletion state** — for soft-deletable entities, whether and when the record was soft-deleted, and by whom.

Audit Log Record Specification (authoritative, resolving the audit's ownership ambiguity)

The **Audit Logs** domain entity itself (distinct from the baseline audit fields every entity carries) is specified as follows:

- **Actor** — the identity that performed the action (a Student's identity, `admin`, `moderator`, or `service`).
- **Action** — a stable, defined identifier for what occurred (e.g., a permission change, a deletion, an admin override) — not free text.
- **Target** — a polymorphic reference (per Section 5) to the entity the action was performed on, plus a discriminator for which domain it belongs to.
- **Timestamp** — see Section 10.
- **Context** — structured, non-sensitive metadata relevant to the action (e.g., what changed), explicitly excluding secrets, full message content, or anything `SECURITY_ARCHITECTURE.md`'s Secrets Management rules would prohibit logging.

- **Severity/category** — a classification aligned with the Bug/Incident severity language used in `TESTING_STRATEGY.md` and `OPERATIONS_ARCHITECTURE.md`, so audit records can be triaged consistently with the rest of the system's severity vocabulary.

`SECURITY_ARCHITECTURE.md` and `API_ARCHITECTURE.md` should, on their next revision, cite this specification rather than continue to independently describe audit log behavior, per `MASTER_ARCHITECTURE.md` Section 9.

10. Timestamp Conventions

- All timestamps are stored in UTC, without exception; any local-time presentation is a UI-layer concern (`UI_DESIGN_SYSTEM.md`), never a storage concern.
 - `created_at` -equivalent and `updated_at` -equivalent concepts are present on every entity per Section 9, set automatically rather than relying on application code to remember to set them.
 - Domain-specific timestamps (e.g., a Hackathon's registration-opens time, a Message's edited-at time) are named for what they represent, not reused generically.
-

11. Versioning Strategy

- Entities whose historical state matters beyond simple audit trail (e.g., a Project's showcase content at time of hackathon submission, which should not silently change after judging begins) carry an explicit version/snapshot concept, distinct from ordinary "last updated at" tracking.
 - Most entities do not need full versioning — versioning is applied deliberately per-domain where a real requirement exists (e.g., Hackathon submission integrity), not as a blanket default, to avoid unnecessary storage and complexity.
 - Schema versioning (i.e., migrations) is covered under Section 1's normalization principle and enforced through the migration process; this section concerns row-level data versioning, a distinct concept.
-

12. Indexing Strategy

- Every foreign key column is indexed by default, since foreign keys are also the most common join/filter pattern.
 - Every field used in an allow-listed filter or sort (per `API_ARCHITECTURE.md`'s Performance section) is indexed deliberately, matched to the specific domain's known access patterns — not indexed speculatively.
 - Composite indexes are used where a domain's access pattern genuinely filters/sorts on multiple columns together (e.g., Hackathon submissions filtered by hackathon and sorted by submission time).
 - High-cardinality, frequently-searched text fields (e.g., Skills, Search-domain content) use an indexing approach suited to text search, distinct from standard equality/range indexes, consistent with the Search domain's ownership (Section 21).
 - Index strategy is revisited whenever Performance Testing (`TESTING_STRATEGY.md`) identifies a query pattern not currently served well — indexing is treated as a living, monitored concern, not a one-time setup task.
-

13. Partitioning Strategy

- No domain is partitioned by default at initial scale; partitioning is introduced deliberately once a specific table's volume or access pattern justifies it, consistent with the one-million-student scalability target (Section 24).
- The domains most likely to require partitioning first, based on expected write volume and natural time-based access patterns, are: **Messages** (high volume, naturally time-ordered), **Audit Logs** (permanent retention, ever-growing), **Notifications** (high volume, naturally aged out), and **Activity Feed** entries (derived, high volume, naturally time-ordered).
- Where partitioning is introduced, time-based partitioning (e.g., by creation period) is the default strategy for the domains above, since their access patterns are predominantly recency-biased.
- Partitioning strategy is an operational/scaling decision, not a schema-shape decision — it must not change any entity's relationships or foreign

key structure, only its physical storage organization.

14. Data Lifecycle

Each domain's lifecycle is specified in its entry in Section 4. At a system level, the common lifecycle stages are:

1. **Creation** — validated at the API layer (`API_ARCHITECTURE.md`) before a record is written, with audit fields (Section 9) populated automatically.
 2. **Active use** — read/write access governed by RLS (Section 19) and API-layer authorization together.
 3. **Soft deletion** (where applicable) — the default end state, per Section 8, preserving referential integrity and audit history.
 4. **Retention-governed hard deletion** — the final stage, per Section 15, for entities and record classes where indefinite retention is not appropriate.
-

15. Data Retention Policy

- **Student-generated content** (Profiles, Projects, Messages, etc.) is retained while the account is active, and for a defined grace period after a soft-delete or account-deletion request, after which it is hard-deleted — balancing the compliance commitments in `SECURITY_ARCHITECTURE.md` (right to deletion) against legitimate short-term recovery needs (e.g., accidental deletion).
- **Audit Logs** are retained permanently by design — they are the one domain explicitly exempted from routine hard deletion, since their entire purpose is a durable historical record. Any exception (e.g., legally mandated purge) is handled as an explicit, audited exception, not routine lifecycle.
- **Notifications** and **Activity Feed** entries are retained only for a bounded, relatively short window, since their value is time-sensitive and their volume is high (see Section 13's partitioning note).
- **Bookmarks** are retained until explicitly removed by the student, with no additional grace period, consistent with their hard-delete-on-unsave lifecycle (Section 8).

- Specific numeric retention windows (e.g., exact grace-period day counts) are deferred as an explicit, flagged placeholder pending confirmation from `MASTER_GOVERNANCE_AMENDMENT_001.md`'s eventual compliance guidance — this document defines which category each domain falls into, not the exact number, consistent with this workstream's established practice of flagging rather than inventing precise figures without a source.
-

16. Backup and Recovery Strategy

- Full backup scheduling, restore process, and Recovery Point/Time Objectives are owned by `DEPLOYMENT_ARCHITECTURE.md`, per `MASTER_ARCHITECTURE.md` Section 9 — this document does not restate them.
 - This document's contribution to backup/recovery is ensuring the schema itself is backup-friendly: no domain relies on data that cannot be captured in a standard database backup (e.g., no critical state held only in ephemeral cache or in-memory structures without a durable source of truth here).
-

17. Multi-Tenant Considerations

HackNest is architected as a **single-tenant system with internal partitioning by affiliation**, not a multi-tenant SaaS platform serving isolated customer organizations:

- Universities and Organizations are first-class entities within one shared database, not separate tenant databases — a student's cross-university team formation (e.g., a multi-campus hackathon team) is a legitimate, expected use case that a hard multi-tenant boundary would prevent.
- RLS (Section 19) provides the isolation that matters — access is scoped by identity and relationship (a student's own data, a team's shared data), not by a rigid tenant boundary.
- If HackNest ever needs to support genuinely isolated institutional deployments (e.g., a university wanting its own fully isolated instance),

that would be a distinct architectural decision requiring a new document and is explicitly out of scope for the current model.

18. Security Boundaries

- The database layer's security boundary is enforced by RLS (Section 19), cooperating with, never substituting for, the API-layer authorization defined in `API_ARCHITECTURE.md` and the broader posture in `SECURITY_ARCHITECTURE.md`, per the Defense in Depth principle in `MASTER_ARCHITECTURE.md` Section 2.
 - The Service Role (which bypasses RLS) is used only by trusted server-side, `service`-identity operations, never exposed to any client context, consistent with `SECURITY_ARCHITECTURE.md`'s Service Role usage rules.
 - No domain's schema is designed assuming client-side enforcement of any security boundary — the database must be safe even if every other layer were somehow bypassed.
-

19. Row Level Security Ownership Model

- RLS policy **design** (which domains are protected, what the access rule conceptually is) is owned entirely by this document, per `MASTER_ARCHITECTURE.md` Section 9.
- RLS policy **enforcement mechanics** (the Supabase-specific policy implementation) is implementation, and explicitly out of scope for this document — that occurs during the relevant `IMPLEMENTATION_ROADMAP.md` phase, not here.
- **General model:** every domain's entities are protected by a policy scoped to identity ownership (Section 3) and role (Section 5) — e.g., a Student can read/write their own Profile and Settings; a Team Member can read/write within their Team's scope per their role; an `admin` or `service` identity has broader access as explicitly defined per-domain, never implicitly.
- **Public read exceptions** are narrow and explicit: Universities (directory), published Hackathon listings, and public-visibility Profiles/Projects/Teams (per each domain's own visibility setting) may

be read without authentication; nothing is public by default or by omission.

- **Write access** is never granted publicly/unauthenticated for any domain.
-

20. File Storage Ownership

- The **Files** domain (Section 4) owns file *metadata*; binary object storage itself is infrastructure, owned by `DEPLOYMENT_ARCHITECTURE.md`'s Infrastructure section (Object Storage).
 - File storage **categories** (avatars, cover images, project media, documents, organization assets) are defined here as they map to the domains that use them (Student Profiles, Projects, Organizations), while the **security controls** governing uploads (allowed types, size limits, scanning, metadata stripping) are owned by `SECURITY_ARCHITECTURE.md`'s File Upload Security section, per `MASTER_ARCHITECTURE.md` Section 9 — this document does not restate those controls, only the data-ownership boundary.
 - Every file metadata record's polymorphic owner reference (Section 5) determines both its RLS scope (Section 19) and its retention category (Section 15).
-

21. Search Indexing Ownership

- The **Search** domain (Section 4) owns the derived, query-time index and the composition contract for cross-domain queries — it never owns or becomes an alternate source of truth for any field.
 - Source domains (Students, Teams, Projects, Hackathons, Organizations) remain the authoritative source for their own data; the Search domain's index is kept eventually consistent with source-domain writes.
 - Search index entries respect the same RLS-equivalent visibility rules as their source domain — a private Team never appears in search results, enforced at the same authorization boundary as direct access, not as a separate, potentially-inconsistent rule.
-

22. Analytics Data Ownership

- The **Analytics** domain (Section 4) owns only aggregate, computed metrics — it never stores individually-identifying raw event data of its own; it computes over source domains' existing data.
 - This ownership model is the direct data-layer implementation of the Privacy by Design principle in `SECURITY_ARCHITECTURE.md` and `MASTER_ARCHITECTURE.md` Section 13 — aggregation happens at query/compute time or into genuinely anonymized aggregate tables, never into a duplicated, identifiable copy of the source data.
 - Any future requirement for individually-identifiable analytics (e.g., a student's personal activity history, which is a legitimate feature) is understood to be a Feed/Profile-domain concern, not an Analytics-domain one — Analytics is exclusively aggregate by definition.
-

23. AI Data Ownership

- Data generated through the AI Builder Workspace (Section 4, AI Builder Profiles) — workspace session state and generated-artifact metadata — is owned by the AI Builder Profiles / Workspace data boundary, distinct from the underlying AI model invocation itself (which is an API/service concern owned by `API_ARCHITECTURE.md`'s AI Builder Workspace domain, not a data-ownership concern here).
 - Generated content that a student chooses to publish (e.g., into a Project) transitions ownership to the target domain (Projects) at that point — the Workspace does not retain a competing, indefinitely-authoritative copy once content has been published elsewhere.
 - AI-generated content remains subject to the same RLS, retention, and audit rules as any other student-owned data — it is not treated as a special, less-protected category.
-

24. Future Scalability Considerations

- The schema and relationship model above are designed to remain valid at the one-million-student target established in

`MASTER_ARCHITECTURE.md` Section 15 — normalization, UUID keys, and deliberate indexing (Section 12) are chosen specifically because they degrade gracefully rather than requiring a redesign at scale.

- The domains flagged for likely future partitioning (Section 13) are the ones expected to be the first practical scaling pressure points, based on write volume and access pattern, not guesswork.
 - Read-heavy, low-volatility domains (Universities, Skills, published Hackathon listings) are natural candidates for caching at the API layer (per `API_ARCHITECTURE.md`'s caching strategy) as an additional scaling lever beyond the database layer itself.
 - Horizontal read scaling (read replicas) and connection pooling are anticipated as the first infrastructure-level scaling responses before any structural schema change is considered necessary — this document's role is to ensure the schema doesn't block those responses, not to prescribe the infrastructure mechanics themselves (owned by `DEPLOYMENT_ARCHITECTURE.md`).
-

Definition of Done

- [x] Architecture and specification only — no SQL, no migrations, no Supabase code, no Prisma schema, no implementation.
 - [x] Follows `README.md` and `MASTER_ARCHITECTURE.md` exactly.
 - [x] All 24 required topics covered.
 - [x] Every domain entry includes Owner, Purpose, Relationships, Lifecycle, and Dependencies.
 - [x] Resolves the audit log ownership ambiguity identified in the Architecture Audit.
-

Validation Report

Internal Consistency

Every domain listed in Section 4 is referenced consistently throughout the rest of the document (relationships in Section 5, RLS model in Section 19, retention category in Section 15). No domain is introduced in one section

and contradicted or omitted in another. The Audit Log record specification (Section 9) is defined exactly once and referenced, not restated, wherever else audit logging is mentioned (Sections 4, 18).

Consistency with `README.md` and `MASTER_ARCHITECTURE.md`

This document occupies exactly the position `MASTER_ARCHITECTURE.md` Section 9 assigned it (sole owner of data shape, relationships, RLS strategy, storage/search/analytics/AI data ownership, and the audit log specification), and uses the canonical role set frozen in that document's Section 5 without redefining it. Confirmed consistent.

Audit Findings Resolved by This Document

Audit Finding	Resolution
Section 1, Item 1 — Missing <code>DATABASE_ARCHITECTURE.md</code> (Critical)	Resolved: this document now exists and is authoritative.
Section 2 — Assumption-chain risk (documents referencing an inferred, never-finalized domain list)	Resolved: Section 4 is now the actual, canonical domain list. Documents that previously inferred this list (<code>API_ARCHITECTURE.md</code> , <code>SECURITY_ARCHITECTURE.md</code> , <code>TESTING_STRATEGY.md</code> , <code>DEPLOYMENT_ARCHITECTURE.md</code> , <code>IMPLEMENTATION_ROADMAP.md</code>) were checked against it during this authoring pass: no material naming conflicts were found — the domain names used across those documents (Teams, Projects, Hackathons, Organizations, Chat, Messages, Notifications, Files, Admin, Analytics, Audit Logs, Settings, etc.) match this document's Section 4 exactly, since they were themselves inferred consistently from the same original request in this workstream. This is a favorable outcome, but those documents should still be formally revised to cite this document going forward rather than continuing to infer it.

Audit Finding	Resolution
Section 6 – Audit logging owned by 3 documents	Resolved. Section 9's Audit Log Record Specification is now the single authoritative definition. <code>SECURITY_ARCHITECTURE.md</code> and <code>API_ARCHITECTURE.md</code> should cite it on their next revision instead of independently describing audit log behavior.
Section 7 – Phase 003 cannot be implemented safely (Critical)	De-risked, not yet fully closed. Phase 003 now has an authoritative domain, relationship, RLS-model, and key-strategy specification to build from. Full closure still requires <code>IMPLEMENTATION_ROADMAP.md</code> to be formally reconciled against this document (a roadmap-editing task, out of scope for this specification-only document per current instructions) and requires <code>PHASE_TEMPLATE.md</code> / <code>CODING_STANDARDS.md</code> for full Phase 001/003 tooling conventions.
Section 9 – Duplicated RPO/RTO placeholder	Not owned by this document (owned by <code>DEPLOYMENT_ARCHITECTURE.md</code> per <code>MASTER_ARCHITECTURE.md</code> Section 9); Section 16 above defers to it correctly rather than restating it.

Outstanding Dependencies

Exact numeric retention windows (Section 15) and full RLS policy mechanics (Section 19) remain flagged placeholders pending `MASTER_GOVERNANCE_AMENDMENT_001.md` and the actual implementation phase, respectively — consistent with this document's own principle of specifying the model precisely while flagging genuinely undetermined figures rather than inventing them.

Status: Validated — Internally Consistent — Consistent with `README.md` and `MASTER_ARCHITECTURE.md` — Ready to Govern Data-Layer Implementation